



This is not mandatory reading, but here's the code we'll run in the "Streamlit in Snowflake" videos. It may come in handy when you're doing the associated hands-on assignment.

```
# Import Python Packages
```

```
import pandas as pd
import streamlit as st
from snowflake.snowpark.context import get_active_session
import altair as alt
```

```
# Get the Current Credentials
```

```
session = get_active_session()
```

```
# Streamlit App
```

```
st.title(":snowflake: Tasty Bytes Streamlit App :snowflake:")
```

```
st.write(
```

```
    """Tasty Bytes is a fictitious, global food truck network, that is on a mission to
    serve unique food options with high quality items in a safe, convenient and cost
    effective way. In order to drive
    forward on their mission, Tasty Bytes is beginning to leverage the Snowflake Data
    Cloud.
```

```
    """
```

```
)
```

```
st.divider()
```

```
@st.cache_data
```

```
def get_city_sales_data(city_names: list, start_year: int = 2020, end_year: int =
2023):
```

```
    sql = f"""
```

```
        SELECT
```

```
            date,
```

```
            primary_city,
```

```
            SUM(order_total) AS sum_orders
```

```
        FROM tasty_bytes.analytics.orders_v
```

```
        WHERE primary_city in ({city_names})
```

```
            and year(date) between {start_year} and {end_year}
```

```

        GROUP BY date, primary_city
        ORDER BY date DESC
    """
    sales_data = session.sql(sql).to_pandas()
    return sales_data, sql

@st.cache_data
def get_unique_cities():
    sql = """
        SELECT DISTINCT primary_city
        FROM tasty_bytes.analytics.orders_v
        ORDER BY primary_city
    """
    city_data = session.sql(sql).to_pandas()
    return city_data

def get_city_sales_chart(sales_data: pd.DataFrame):
    sales_data["SUM_ORDERS"] = pd.to_numeric(sales_data["SUM_ORDERS"])
    sales_data["DATE"] = pd.to_datetime(sales_data["DATE"])

    # Create an Altair chart object
    chart = (
        alt.Chart(sales_data)
        .mark_line(point=False, tooltip=True)
        .encode(
            alt.X("DATE", title="Date"),
            alt.Y("SUM_ORDERS", title="Total Orders Sum USD"),
            color="PRIMARY_CITY",
        )
    )
    return chart

def format_sql(sql):
    # Remove padded space for visual purposes
    return sql.replace("\n        ", "\n")

first_col, second_col = st.columns(2, gap="large")

with first_col:
    start_year, end_year = st.select_slider(

```

```

        "Select date range you want to filter the chart on below:",
        options=range(2020, 2024),
        value=(2020, 2023),
    )
with second_col:
    selected_city = st.multiselect(
        label="Select cities below that you want added to the chart below:",
        options=get_unique_cities()["PRIMARY_CITY"].tolist(),
        default="San Mateo",
    )
if len(selected_city) == 0:
    city_selection = ""
else:
    city_selection = selected_city
city_selection_list = ("'" + "', '".join(city_selection) + "'") if city_selection else ""

sales_data, sales_sql = get_city_sales_data(city_selection_list, start_year, end_year)
sales_fig = get_city_sales_chart(sales_data)

chart_tab, dataframe_tab, query_tab = st.tabs(["Chart", "Raw Data", "SQL Query"])
chart_tab.altair_chart(sales_fig, use_container_width=True)
dataframe_tab.dataframe(sales_data, use_container_width=True)
query_tab.code(format_sql(sales_sql), "sql")

```